

AI ENGINEER STUDY GUIDE · COMPANION

The Evolution of Language Models

From n-grams to reasoning models

A guided tour of two decades of language modeling

HOW THIS GUIDE READS

Each chapter follows the same shape:

What the model could do → Where it broke down → What came next

The story is chronological. Every breakthrough was a response to a specific limitation. Read in order and you'll see why the field moved the way it did.

Introduction

Today's large language models — Claude, GPT-4, Gemini, Llama — feel like a sudden arrival. They aren't. They're the result of two decades of incremental research, each step a response to specific failures of the step before.

This guide tells that story. It's not a survey of every paper. It's a *narrative* — what worked, what broke, what came next. Each section follows the same shape:

- 1. The model and what it could do.** A clear picture of how it worked and what it achieved at the time.
- 2. A concrete example.** What the model would actually produce when you used it.
- 3. The limitation that hurt.** Where the approach broke down, and why researchers couldn't ignore it.
- 4. What came next.** The breakthrough that addressed the limitation — and set up the next chapter.

Read in order. Each chapter assumes the previous one. By the end you'll have a working mental model of why language modeling moved the way it did, and where it's heading.



Figure 0.1 — The seven eras of language modeling. Each transition was driven by a specific limitation of the prior approach.

2003 — 2013 Statistical Era

Count word combinations. Predict the next word from a short context.

Chapter 1. n-gram Language Models

Before neural networks took over, language models were built from word counts. If you wanted to model a language, you took a huge corpus of text, counted how often each sequence of N consecutive words appeared, and used those counts to predict the next word.

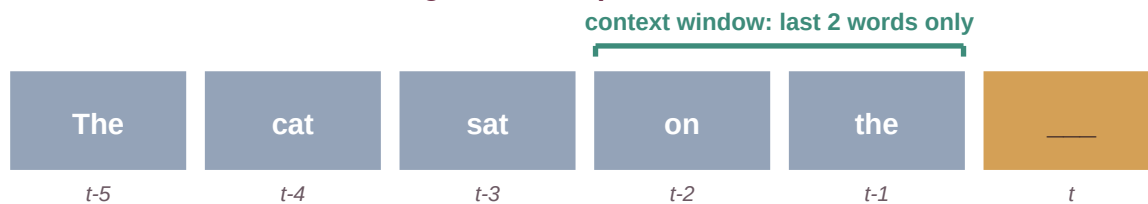
How they worked

An **n-gram model** is a model that predicts the next word using only the previous N-1 words. A *trigram* model (n=3) uses the last 2 words. A *5-gram* model uses the last 4. The prediction is a simple count ratio:

$$P(\text{next word} \mid \text{last 2 words}) = \text{count}(\text{"last 2 words next"}) / \text{count}(\text{"last 2 words"})$$

If "on the floor" appeared 1,200 times in your training corpus and "on the" appeared 10,000 times, then $P(\text{"floor"} \mid \text{"on the"}) = 0.12$.

How a trigram model picks the next word



$$P(\text{next word} \mid \text{"on the"}) \approx \text{count}(\text{"on the X"}) / \text{count}(\text{"on the"})$$

Words like "The", "cat", "sat" — ignored. No memory.

If you saw "on the" → most likely next: "floor", "table", "way", "mat"...

Figure 1.1 — A trigram model looks only at the last two words to predict the next. Everything earlier in the sentence is ignored.

EXAMPLE — A 5-gram model in 2010

Input: "The weather forecast for tomorrow is" → Output (most likely next word): "sunny" / "cloudy" / "rainy". The model could complete common phrases but had no understanding. Given "The capital of France is", a 5-gram model would predict "the" or "a" before "Paris", because "is the" is far more common in text than "is Paris".



WHY IT BROKE DOWN

1. No memory beyond N words. A 5-gram model has no idea what was said 10 words ago. In "The man who walked into the store yesterday afternoon during the rainstorm bought a ____", the model only sees "bought a" — the entire context is invisible.

2. Combinatorial explosion. For a vocabulary of 50,000 words, the number of possible 5-grams is $50,000^5 = 3 \times 10^{22}$. Most never appear in any corpus. The model has to fall back to smoothing tricks to handle unseen sequences.

3. No generalization. If the model has seen "cat sat on mat" but never "dog sat on rug", it treats them as completely unrelated. There's no concept of meaning — just surface-level co-occurrence counts.



WHAT CAME NEXT — NEURAL LANGUAGE MODELS (Bengio et al., 2003 → 2013)

Bengio and colleagues had proposed using neural networks for language modeling back in 2003, but it didn't scale at the time. Around 2010-2013, the idea finally caught on: instead of counting word sequences, learn a **dense vector representation** for each word (a "word embedding") and let a neural network predict the next word from those vectors. Now "cat" and "dog" can have similar vectors, so the model generalizes between them. Word2Vec (2013) made this practical at scale and set off the neural NLP revolution.

2014 — 2017 Sequential Neural Era

Neural networks with memory. Process sequences one token at a time.

Chapter 2. RNNs and the Memory Problem

If n-gram models forgot anything older than N words, the obvious fix was to give the model some *memory*. Recurrent Neural Networks (RNNs) added exactly that. An RNN processes a sequence one token at a time, maintaining a hidden state vector that gets updated at each step. In principle, this hidden state could carry information from anywhere in the sequence.

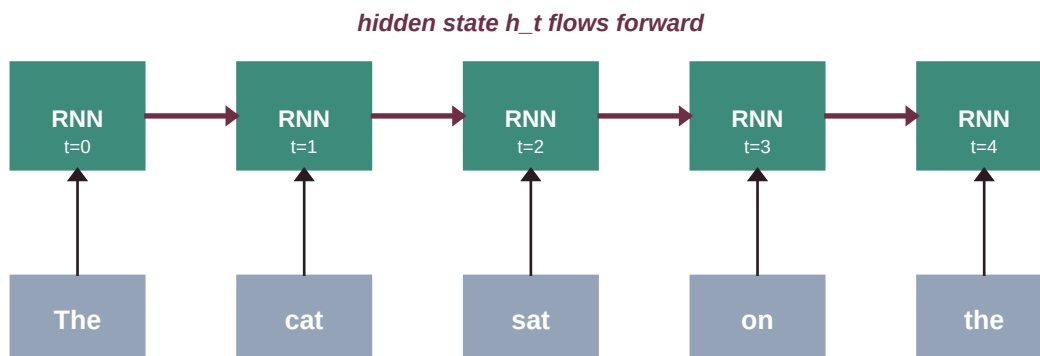
How they worked

At each time step t , the RNN takes the current input word and the hidden state from the previous step, and produces a new hidden state and an output. The recurrence:

$$h_t = \tanh(W_x * x_t + W_h * h_{(t-1)} + b)$$

The hidden state h_t “remembers” something about everything seen so far. The same weights W are reused at every time step, so the model can handle sequences of any length.

An RNN processes a sequence one step at a time



Each step depends on the previous one — cannot parallelize across t .

Figure 2.1 — An RNN “unfolded” across time. Each step receives both a new input and the previous hidden state. This sequential dependency is its strength (memory) and its weakness (no parallelism).

EXAMPLE — A character-level RNN trained on Shakespeare (Karpathy, 2015)

Trained on the complete works of Shakespeare, a small character-level RNN could generate output like: "PANDARUS: Alas, I think he shall be come approached and the day / When little strain would be attain'd into being never fed, / And who is but a chain and subjects of his death..." Coherent enough to look Shakespearean, with proper character names and meter. But meaningless if you read closely — no actual plot, no logical structure.



WHY THEY BROKE DOWN — VANISHING GRADIENTS

Theoretically, RNNs could carry information arbitrarily far. In practice, they couldn't. When you backpropagate through many time steps, the gradient signal gets multiplied by weight matrices over and over. If those products are less than 1, the gradient **vanishes exponentially** — by the time it reaches the start of a long sequence, it's essentially zero. The model can't learn long-range dependencies.

Practical result: RNNs handled maybe 20-50 tokens of useful context. Beyond that, the earlier context might as well not exist. For a model that needs to remember a subject from the start of a long paragraph to agree with a verb at the end, this was fatal.



WHAT CAME NEXT — LSTMs and GRUs

The Long Short-Term Memory (LSTM) network, invented by Hochreiter & Schmidhuber in 1997 but popularized around 2014, solved the vanishing gradient problem with a clever gating mechanism. LSTMs have an extra **cell state** alongside the hidden state, plus three learned gates (input, forget, output) that decide what to keep, forget, or expose. The cell state can carry information across hundreds of time steps without decay.

GRUs (Gated Recurrent Units, 2014) are a simpler variant with just two gates and no separate cell state — cheaper to train, often comparable in quality. By 2015, LSTMs were the standard architecture for everything sequential: machine translation, speech recognition, text generation.

Chapter 3. Seq2Seq and the Bolt-On of Attention

LSTMs were a clear improvement, but real applications needed more than just predicting the next word. Machine translation, for instance, needs to convert an entire sentence in English to an entire sentence in French — different lengths, different word orders. This is a *sequence-to-sequence* problem.

The seq2seq architecture

Sutskever, Vinyals, and Le (2014) introduced the encoder-decoder structure: an **encoder** LSTM reads the input sequence and compresses it into a single fixed-size vector. A **decoder** LSTM takes that vector and generates the output sequence, one token at a time.

This worked surprisingly well for short sentences. Google Translate started switching from phrase-based statistical translation to neural seq2seq around 2016 with dramatic quality improvements.

EXAMPLE — Neural machine translation, 2016

Input: "The agreement on the European Economic Area was signed in August 1992."

Older phrase-based system: "L'accord sur la zone économique européenne a été signé en août 1992." Stilted, sometimes mangled.

Seq2seq system: "L'accord sur l'Espace économique européen a été signé en août 1992." Idiomatic, more fluent. The difference was immediately obvious to bilingual users.

! WHY SEQ2SEQ ALONE BROKE DOWN — THE BOTTLENECK

The encoder compresses the **entire input sentence** into a single fixed-size vector — usually 256 to 1024 dimensions. For short sentences this is fine. For long sentences, it's a brutal information bottleneck.

Quality degraded sharply for sentences longer than ~20 words. The decoder, looking at one summary vector, couldn't tell which part of the source sentence to focus on when generating each output word. The vector had to compress "the subject is plural" and "the verb tense is past" and "the object is in Paris" all into the same 512 numbers.

→ WHAT CAME NEXT — ATTENTION (Bahdanau et al., 2014)

Bahdanau, Cho, and Bengio introduced **attention**: instead of compressing the entire source sentence into one vector, let the decoder "look back" at the encoder's hidden states whenever it generates each output word. At every decoding step, compute a weighted average of *all* encoder states, with weights based on relevance to the current decoding position.

Practical result: long-sentence translation quality recovered. The model could focus on "the European Economic Area" when generating the French equivalent, and ignore other parts of the sentence.

Attention was bolted on to existing LSTM seq2seq. It worked beautifully. But soon someone would ask: if attention is doing the heavy lifting, do we even need the LSTMs?

2017

Transformer Pivot

One paper changed everything: attention is all you need.

Chapter 4. The Transformer

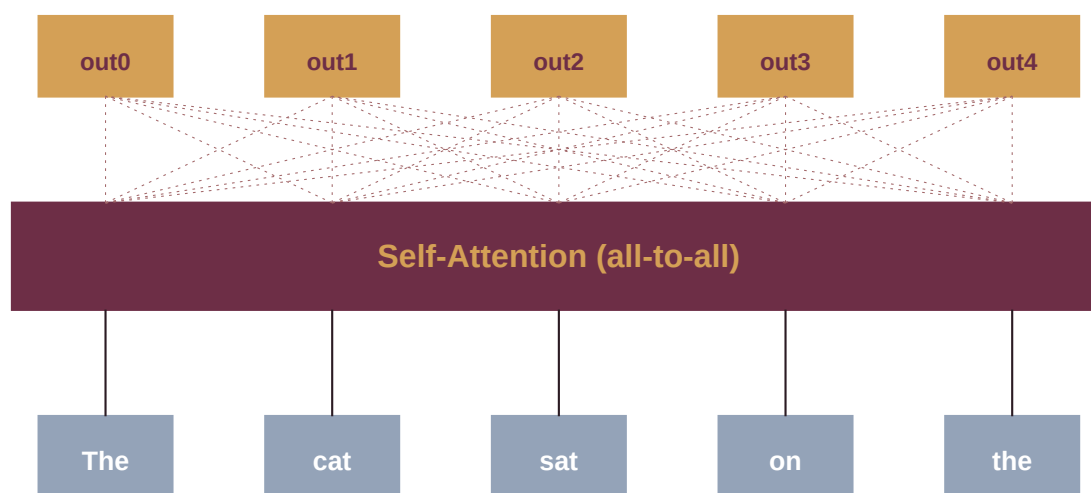
In 2017, Vaswani et al. at Google published *Attention Is All You Need*. The title was the thesis. They removed the recurrent layers entirely and built a model using only attention. The result was the **Transformer**, and it redefined what neural networks could do.

Why this was radical

RNNs and LSTMs were *sequential* by design — each step depended on the previous one. You couldn't parallelize them across time. To process a 100-token sentence, you had to do 100 sequential matrix operations.

The Transformer uses **self-attention**: every token looks at every other token simultaneously. There's no time dimension in the computation. You can compute all positions in parallel on a GPU. For a 100-token sentence, one parallel operation does what RNNs needed 100 sequential operations for.

The transformer processes all tokens in parallel



Every token can attend to every other token — fully parallel computation.

Figure 4.1 — The transformer processes all tokens at once through self-attention. Dashed lines show that every output position can attend to every input position. No sequential dependency means full GPU parallelism.

The three key ideas

1. Self-attention. For each token in the input, compute a Query vector Q , a Key vector K , and a Value vector V . Compare the Query to every Key to get attention weights. Take a weighted sum of the Values. The result is a new representation that incorporates information from across the entire sequence.

2. Multi-head attention. Do the above multiple times in parallel with different learned projections. Each "head" can focus on a different type of relationship — one might track syntactic dependencies, another semantic similarity, another positional patterns.

3. Positional encodings. Since self-attention is order-invariant by default (it doesn't know which token came first), the model adds positional encoding vectors to the input embeddings. This is how the network learns about word order.

EXAMPLE — The original Transformer at machine translation

*On the WMT 2014 English-to-German benchmark, the original Transformer scored 28.4 BLEU — about 2 points better than the best previous models, which were all LSTM-based. More importantly, it trained **3-10x faster** on the same hardware because of the parallelism. Combined with bigger datasets and bigger models, this efficiency unlocked the scaling era.*



WHAT EVEN THE TRANSFORMER COULDN'T DO YET

The 2017 transformer was an architecture, not a general language understanding system. To use it, you trained one from scratch for each task: one transformer for translation, another for summarization, another for question answering. Each needed a labeled dataset and training run.

And while attention solved long-range dependencies *in principle*, the quadratic compute cost (every token attending to every other token) meant you couldn't process arbitrarily long sequences. Practical context windows in 2017 were ~512 tokens — enough for a paragraph, not enough for a chapter.



WHAT CAME NEXT — PRETRAIN + FINE-TUNE

Within a year, researchers realized you could **pretrain** a transformer on a huge unlabeled corpus (just predict missing words from text), then **fine-tune** the resulting model on small labeled datasets for specific tasks. The pretraining would teach the model general language patterns; the fine-tuning would specialize it. This unlocked two parallel research streams: BERT for understanding tasks, and GPT for generation tasks.

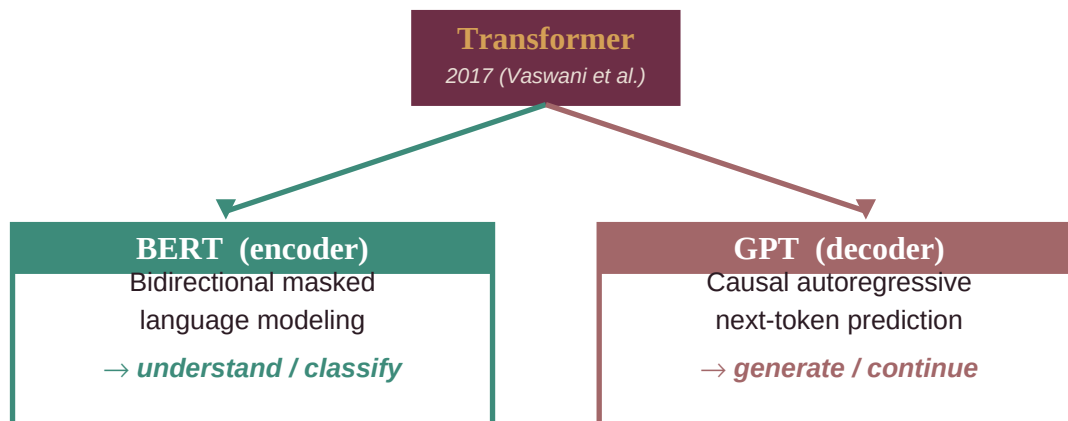
2018 — 2020 Pretrain + Fine-tune Era

One pretrained model, many fine-tuned applications.

Chapter 5. BERT and GPT-1 — The Fork

The transformer architecture had two halves: an encoder (which processes input bidirectionally) and a decoder (which generates output left-to-right). In 2018, two papers split these halves into separate model families.

After the transformer, two different specializations emerged



Same building blocks. Different training objective. Different downstream uses.

Modern LLMs (GPT-4, Claude, Llama)
evolved from this branch

Figure 5.1 — In 2018, the transformer split into two specializations. BERT used only the encoder for understanding tasks. GPT used only the decoder for generation. GPT's lineage led directly to today's LLMs.

BERT (Devlin et al., October 2018)

BERT — Bidirectional Encoder Representations from Transformers — used only the transformer's encoder. Its pretraining task was clever: take a sentence, **mask 15% of the tokens**, and train the model to predict the masked tokens from the surrounding context (both left *and* right). This is called *masked language modeling*.

The bidirectional context made BERT exceptional at understanding tasks: classification, named entity recognition, question answering. To use BERT for a new task, you'd add a small task-specific head on top and fine-tune on a few thousand examples.

EXAMPLE — BERT at GLUE benchmark, 2018

GLUE was the standard set of language understanding tasks (sentiment, entailment, paraphrase detection). Before BERT, the state of the art on the combined GLUE score was about 70%. BERT-Large scored 80.5% immediately on release — a 10-point jump that the field hadn't seen since the deep learning revolution. Suddenly, transfer learning worked for NLP.

GPT-1 (Radford et al., June 2018)

GPT — Generative Pretrained Transformer — used only the decoder half. Its pretraining task was simpler: **predict the next token** given everything before it. Just left-to-right next-token prediction, run on a large corpus of text (BookCorpus, ~7,000 books).

GPT-1 (117M parameters) was decent at generation tasks and could be fine-tuned for classification too, though BERT was generally better for understanding tasks of that era. The real story of GPT was about to start with GPT-2.

**WHY THE FINE-TUNE-FOR-EVERYTHING APPROACH BROKE DOWN**

Pretrain-then-fine-tune was a huge improvement over training from scratch. But it still required labeled data and a training run **for every new task**. Want a model for medical text classification? Get a labeled medical dataset and fine-tune. Want one for legal text? Get a labeled legal dataset and fine-tune. Want one for customer support? Same.

This was a logistical and economic problem. Most companies didn't have labeled datasets for their specific use cases. Hiring labelers was expensive. And each fine-tuned model lived in its own silo — you couldn't combine them or share them across teams without retraining.

**WHAT CAME NEXT — SCALE CHANGES THE GAME**

OpenAI made a bet: what if you didn't need to fine-tune at all? What if you just made the pretrained model much bigger and showed it more text? Maybe it could learn to do tasks just from a natural-language description, without task-specific training. The result was GPT-2 — and the start of the scaling era.

2019 — 2022 Scaling Era

Bigger model. More data. More compute. Emergent capabilities.

Chapter 6. GPT-2 and GPT-3 — Scale as a Discovery

GPT-2, published in February 2019, was structurally identical to GPT-1. Same architecture, same training objective (next-token prediction), same code. The difference: 10x more parameters (117M → 1.5B) and 10x more training data (7k books → 40 GB of web text).

What OpenAI discovered was startling: GPT-2 could do tasks it had never been trained on, just by reading a prompt. Want it to translate? Show it a few English-French pairs and it would try. Want it to summarize? Add "TL;DR" at the end of an article and it would generate a summary. None of this had been explicitly trained — it emerged from sheer scale plus a diverse training corpus.

EXAMPLE — GPT-2 doing zero-shot translation

Input prompt: "The English sentence 'The cat sat on the mat' translates to French as:"

GPT-2 output: "Le chat s'est assis sur le tapis."

GPT-2 wasn't trained to translate. There was no parallel English-French dataset in its training. But the web text had occasional bilingual sentences, English discussions of French, etc. From this messy data, it had picked up enough structure to attempt the task. Quality was mediocre but the fact that it worked at all was new.

GPT-3 (June 2020) — the scale jump

OpenAI scaled again. GPT-3 was 175 billion parameters — 100x larger than GPT-2 and roughly 1,000x larger than GPT-1. Trained on ~570 GB of filtered internet text.

What emerged at this scale shocked even the researchers: **in-context learning**. Give GPT-3 a few examples of a task in the prompt, and it would perform the task on new inputs — without any gradient updates, without any fine-tuning. The "learning" happened entirely during inference, by reading the examples in the context window.

Capabilities emerged with scale, not new architectures

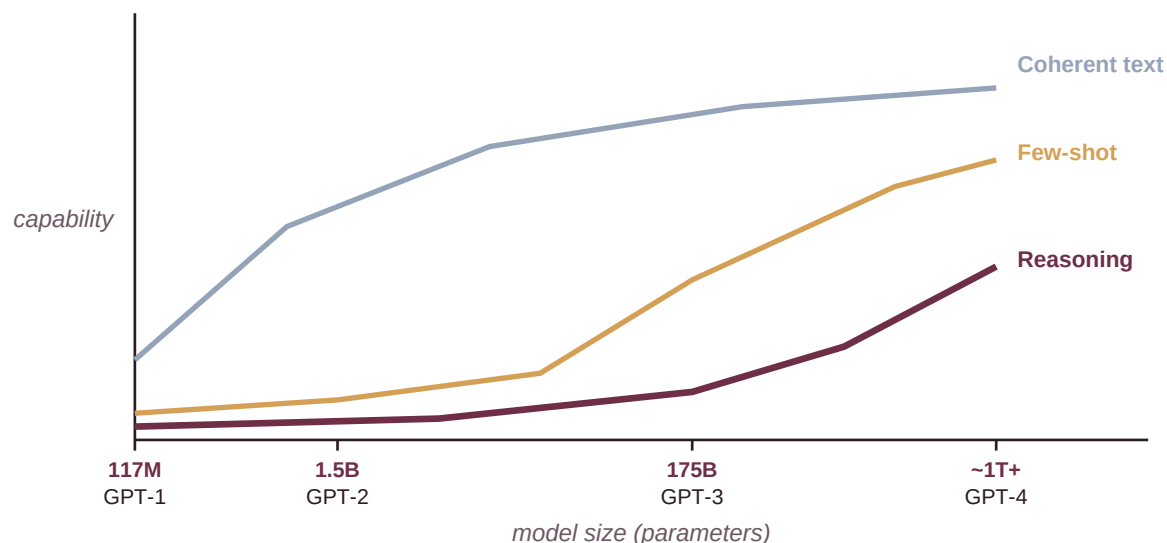


Figure 6.1 — Different capabilities required different scales. Coherent text generation emerged early. Few-shot learning appeared around GPT-3 scale. Multi-step reasoning required even more.

EXAMPLE — GPT-3 doing in-context learning (zero gradient updates)

Prompt: "Translate Klingon words to English.

tlhIngan → Klingon

Hol → language

jagh → enemy

Heghlu'meH →"

GPT-3 output: "to die"

GPT-3 was never trained on Klingon. It "learned" the pattern from three examples in the prompt. This wasn't transfer learning in the traditional sense — no weights changed. The model just inferred the rule from the context. This capability was, frankly, magical when it appeared.



WHY GPT-3 STILL WASN'T "USEFUL" THE WAY CHATGPT WOULD BE

GPT-3 was extraordinarily capable but extraordinarily awkward. It was trained to **predict the next word in web text** — not to be helpful, not to follow instructions, not to refuse harmful requests.

Practical problems:

- Ask GPT-3 a question and you'd often get more questions back (because questions tend to be followed by more questions on web pages, not answers).
- Ask it to write an essay and it might continue with "...is the title of the essay I will write" because that pattern appeared in its training data.
- It would happily generate dangerous content, hallucinate confidently, or produce gibberish without warning.

GPT-3 was a brilliant student who'd read the entire internet but had no instructor to teach them how to actually help anyone.



WHAT CAME NEXT — RLHF AND INSTRUCTION-FOLLOWING

The missing piece was a way to align the model with what humans actually want. OpenAI's solution, building on earlier alignment research: train the model with human feedback. Show it pairs of responses and have humans rank which is better. Use those rankings to train a **reward model**, then use reinforcement learning to make the LLM produce responses that score high under the reward model. This is Reinforcement Learning from Human Feedback — RLHF — and it transformed GPT-3 into ChatGPT.

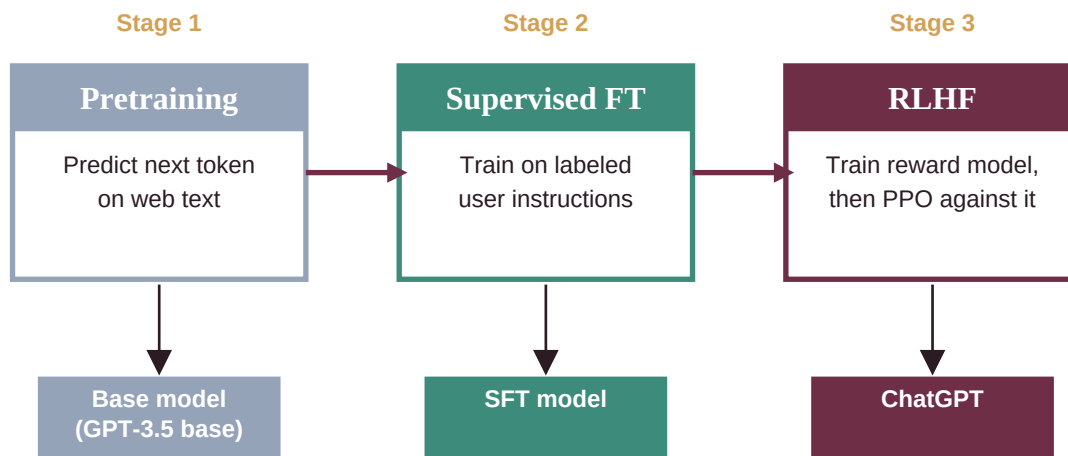
2022 — 2024 Instruction Era

Models that follow instructions. Chat interfaces. The mainstream arrives.

Chapter 7. InstructGPT, ChatGPT, and RLHF

In January 2022, OpenAI published the InstructGPT paper (Ouyang et al.) describing how to make a language model actually *useful*. The technique had three stages.

How ChatGPT was made: the three-stage RLHF pipeline



The shift: from "continue the text" → "do what the human meant"

Figure 7.1 — The RLHF pipeline. Pretraining gives the model raw language ability. Supervised fine-tuning teaches the format of instruction-following. RLHF tunes the model to match human preferences for what makes a "good" response.

Stage 1 — Pretraining

Start with a pretrained GPT-style model. This is the GPT-3 we already had — fluent but unhelpful.

Stage 2 — Supervised fine-tuning (SFT)

Hire human labelers. Have them write thousands of high-quality examples of instruction → ideal response. Fine-tune the pretrained model on these examples using standard supervised learning.

Result: a model that has learned to produce responses in the "helpful assistant" format. Already much better than raw GPT-3 at following instructions. But still imperfect — the model didn't always know what made a response good versus mediocre.

Stage 3 — RLHF (Reinforcement Learning from Human Feedback)

Give the SFT model a prompt. Have it generate several different responses. Have human labelers rank them from best to worst. Use those rankings to train a separate **reward model** — a neural network that predicts the human ranking from any response.

Then use reinforcement learning (specifically PPO — Proximal Policy Optimization) to tune the SFT model: generate responses, score them with the reward model, and adjust the weights to produce higher-scoring outputs.

Result: a model that doesn't just follow instructions, but follows them in a way humans prefer. More polite, more thorough, more careful about ambiguity, more likely to refuse harmful requests.

EXAMPLE — GPT-3 vs ChatGPT on the same prompt

Prompt: "Explain the concept of recursion to a 10-year-old."

GPT-3 (raw): "Recursion is a method where the solution to a problem depends on solutions to smaller instances of the same problem. It is widely used in computer science..." Technically correct, useless for a 10-year-old.

ChatGPT: "Imagine you have a set of Russian nesting dolls. To find the smallest doll, you open the biggest one, then the next, then the next — each step is the same kind of action, just on a smaller doll. Recursion in programming is when a function calls itself to solve smaller pieces of the same problem..." Same accuracy, dramatically better delivery. This is what RLHF bought.

Why it spread so fast

ChatGPT launched November 30, 2022, and hit 100 million users in two months — the fastest consumer software adoption in history at the time. The architecture underneath was the same transformer family that had been around since 2017. The scale was already there from GPT-3 in 2020. The breakthrough was that for the first time, the model would *actually do what you asked* in a conversational way that anyone could use. The interface caught up with the capability.



WHAT THE INSTRUCTION ERA STILL COULDN'T DO

RLHF-tuned LLMs in 2022-2023 had three persistent weaknesses:

- 1. Hallucination.** The model produced plausible-sounding text whether or not it was factually correct. It would confidently cite papers that didn't exist, describe events that never happened, and provide wrong but well-written answers.
- 2. Poor reasoning.** Anything requiring multi-step logical inference — math problems, complex code debugging, careful argument analysis — was unreliable. The model would often produce confident wrong answers on problems that required actual thinking.
- 3. Limited context.** Early ChatGPT had a 4,000-token context window. You couldn't feed it a whole book, or a large codebase, or weeks of conversation history. Important information would scroll out of view.



WHAT CAME NEXT — BIGGER, MULTIMODAL, MORE CAPABLE

The next two years brought a flurry of improvements rather than a single breakthrough: GPT-4 (multimodal: text + images), Claude with 100K-token context windows, Gemini, Llama 3, and many others. Each made the models more capable, more reliable, more multimodal, and longer-context. But the fundamental architecture was still the RLHF-tuned transformer of 2022 — just bigger, with more training, with better data. The reasoning problem remained unsolved. Until the next era arrived in 2024.

2024 — today Reasoning Era

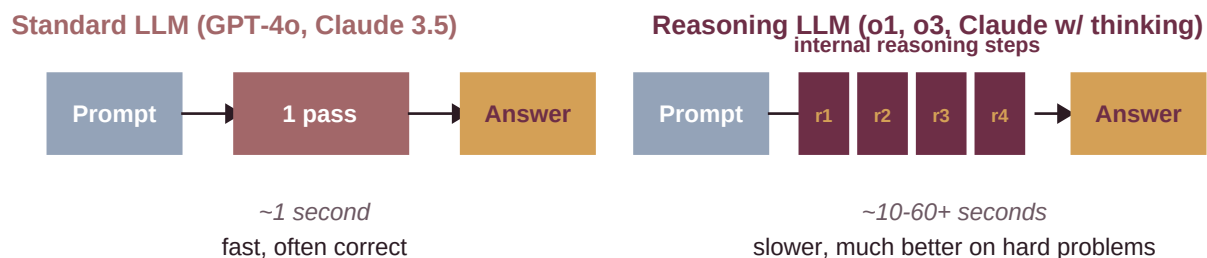
Models that think before they answer. Test-time compute scaling.

Chapter 8. Reasoning Models — Thinking Before Answering

For a long time, AI compute was understood in two phases: **training compute** (huge, one-time, done in a data center) and **inference compute** (small, per-request, ideally fast). The way to make models smarter was to spend more training compute. Inference was just “run the trained model.”

In September 2024, OpenAI released o1 (“o” for “O”-rion, then later renamed). The bet: take a strong base model and train it specifically to produce long internal reasoning chains before answering. At inference time, the model would think for seconds, minutes, or even longer — generating tokens of reasoning that the user never sees — before producing the final answer.

Reasoning models trade more inference compute for higher quality



Same architecture family. Different inference strategy: think before answering.

Best for math, code, multi-step problems. Overkill for chitchat.

Figure 8.1 — Reasoning models spend much more inference compute than standard LLMs. They generate long internal reasoning chains (“thinking tokens”) that the user doesn’t see, then produce a final answer. The trade-off: 10-100× slower, but dramatically better on problems requiring multi-step inference.

What's new technically

The architectural changes from earlier LLMs were modest. The training process changed substantially:

- 1. Reasoning-focused training data.** Curated datasets of math problems, code, logical puzzles — each paired with detailed step-by-step solutions.
- 2. Reinforcement learning at scale.** Train the model with RL signals based on whether the final answer is correct, not just whether the response looks good to a human. The model learns to *actually solve* the problem.
- 3. Hidden chain-of-thought.** The model generates long internal reasoning (often 10× longer than the final answer), explores multiple approaches, backtracks from mistakes, then condenses everything into a final

user-facing response.

4. Test-time compute scaling. The same model can be allocated more or less thinking time. With more compute, it tries more approaches and verifies more carefully. Performance improves smoothly with thinking time on hard problems.

EXAMPLE — A reasoning model on a hard math problem

Problem: "Find the smallest positive integer n such that $n^3 + 2n^2 + n$ is divisible by $6n + 18$."

Standard LLM: tries direct factoring, gets confused, produces a wrong answer confidently.

Reasoning model (Claude with extended thinking, or o1): thinks for 45 seconds. Internally: factors the numerator, tries polynomial long division, checks small values, considers the algebraic identity, verifies the answer satisfies the divisibility condition. Then outputs the answer with a short explanation.

On benchmarks like AIME (advanced math), reasoning models jumped from ~12% (standard GPT-4) to 75%+ accuracy. On competitive programming, similar gains. The improvement is most dramatic exactly where standard models hallucinated most.

What changed for the field

Reasoning models broke an assumption that had held since the start: that the quality of an LLM's answer was set at training time. With reasoning models, **you can pay more at inference time** to get a better answer. This opens new product designs: spend pennies for quick answers, dollars for hard problems. It also matters for safety — critical decisions can get more thinking budget.

By 2025, every major lab had reasoning variants: OpenAI's o1, o3, o4-mini; Anthropic's Claude with extended thinking; Google's Gemini Deep Think; DeepSeek R1; Qwen QwQ. The reasoning paradigm became part of the standard LLM toolkit.



WHAT REASONING MODELS STILL DON'T SOLVE

Cost and latency. Reasoning costs 10-100× more per request. Many use cases can't tolerate the latency or budget. For chatbot small-talk, reasoning is wasteful.

Unfaithful reasoning. The internal chain-of-thought sometimes doesn't match how the model actually reached its answer. The user-facing trace can look principled while the underlying computation took shortcuts. This is an active research problem in AI interpretability.

Reasoning isn't general intelligence. These models are strong on math, code, and well-defined puzzles. They're not obviously better on open-ended creative tasks, emotional nuance, or ill-defined real-world judgment calls. More inference compute helps where the problem has a verifiable correct answer.

Chapter 9. Where We Are Now

By 2025, the LLM landscape has settled into a recognizable shape. Different model classes serve different purposes, and the differences are getting clearer.

9.1 The current taxonomy of models

General-purpose chat models

Examples: GPT-4o, Claude Sonnet, Gemini Flash

Fast, multimodal, broadly capable. The default for most user-facing applications. Optimized for latency and cost.

Frontier reasoning models

Examples: o3, Claude Opus with thinking, Gemini Deep Think

Slow, expensive, dramatically better on hard problems. Used when correctness matters more than speed.

Small efficient models

Examples: Gemini Nano, Claude Haiku, Llama 3.2 1B, Phi

Run on phones or modest hardware. Good enough for many narrow tasks. Used for on-device features and edge deployment.

Open-weight models

Examples: Llama, Mistral, DeepSeek, Qwen, Gemma

Downloadable weights you can fine-tune, host yourself, or modify. The backbone of the open-source ecosystem. Quality keeps closing the gap with frontier models.

9.2 Architectural trends since the transformer

Mixture of Experts (MoE)

Instead of one giant dense network, MoE models have many smaller "expert" subnetworks plus a gating mechanism that routes each input to a few experts. Effective parameter count is much higher than active parameters at inference. Mixtral, DeepSeek-V3, GPT-4 (reportedly) all use MoE. Lets you scale model capacity without proportional inference cost.

Long context windows

Early ChatGPT had 4K tokens of context. Today: Claude handles 200K+ tokens, Gemini handles 1M+. This required new tricks: FlashAttention (memory-efficient attention), sliding window attention, ring attention, and others. You can now feed an LLM an entire book, a long codebase, or hours of meeting transcripts.

Multimodality

Modern LLMs handle text, images, audio, and video natively. The underlying approach (from CLIP, LLaVA, GPT-4V): use a Vision Transformer to encode images into the same token space as text, then feed both to the same LLM. Tomorrow's models will treat all modalities as first-class — generate images and audio as easily as text.

Tool use and agents

LLMs can call external tools: web search, code interpreters, databases, specialized models. Models trained for tool use can chain many calls to accomplish complex tasks autonomously — the emerging "agent" paradigm. Claude Code, GPT operator, Claude in Chrome all exemplify this direction.

9.3 What's likely next

The pattern of the last two decades — each leap addressing a specific limitation of the previous one — will continue. Plausible next themes:

Continuous learning. Today's LLMs are frozen after training. Future models may update from interaction or new information without full retraining.

Verified reasoning. Reasoning that can be checked for correctness, either by the model itself or by external verifiers. Reduces hallucination and unfaithful reasoning.

Memory and personalization. Models that remember a specific user across sessions, learn their preferences, build a working model of their context — without privacy compromises.

True multimodal generation. Models that handle text, images, audio, video, 3D, and code in a single unified framework, generating any modality from any input.

Smaller specialist models. As capabilities mature, much can be moved to smaller models that run locally, cheaply, and privately. Edge AI as the default, cloud AI as the exception.

Chapter 10. Why This History Matters

10.1 Three patterns that have held

Pattern 1 — Scale unlocks capabilities

Most of the dramatic moments in LLM history weren't new architectures — they were the same architecture at a larger scale. GPT-2 to GPT-3 was 10× more parameters. GPT-3 to GPT-4 was reportedly similar. Each jump revealed capabilities the previous scale didn't have.

When evaluating a new technique, ask: does it work because it's genuinely smarter, or because it would work with enough scale anyway?

Pattern 2 — Bottlenecks drive innovation

Every era's breakthrough was a response to the prior era's bottleneck. n-grams couldn't remember → RNNs. RNNs couldn't parallelize → transformers. Transformers needed labels for every task → pretraining. Pretrained models couldn't follow instructions → RLHF. RLHF models couldn't reason → reasoning models.

Pay attention to what today's frontier models *can't* do. That's where the next era will live.

Pattern 3 — The interface matters as much as the model

GPT-3 had the capabilities of ChatGPT for two years before ChatGPT existed. What made the difference wasn't a model improvement — it was wrapping the model in a chat interface and tuning it to be helpful.

Most of an LLM product's value comes from the integration: how it's prompted, what tools it can call, how it presents results. Don't underestimate this work — it can multiply the value of an unchanged base model.

10.2 What to remember as a practitioner

Whether you're building products with LLMs, fine-tuning them, or interviewing for AI roles, this history is the conceptual backbone you need:

1. Today's LLMs are decoder-only transformers, descended from GPT-1. Understanding the transformer is the single most important piece of architectural knowledge in the field.
2. Pretraining gives raw capability. SFT teaches format and instruction-following. RLHF aligns to human preferences. Reasoning RL teaches the model to think before answering. Most modern LLMs go through all four.
3. In-context learning is the model's ability to learn from examples in the prompt without any training. This is why prompt engineering works.
4. Fine-tuning (especially with LoRA — see the companion guide) lets you specialize a model for your task without retraining from scratch. This is how most production LLM customization happens.
5. Test-time compute is the new dimension. Reasoning models trade speed and money for quality. Choose the right tradeoff per task.

10.3 Reading list to dig deeper

Word2Vec (Mikolov et al., 2013)	Showed that dense word vectors capture meaning. The conceptual foundation for everything that came after.
Seq2Seq with attention (Bahdanau et al., 2014)	Introduced attention as a mechanism. The direct precursor to transformers.
Attention Is All You Need (Vaswani et al., 2017)	The transformer paper. Required reading. Short, dense, foundational.
BERT (Devlin et al., 2018)	The bidirectional encoder. Defined the pretrain+fine-tune paradigm for understanding tasks.
Language Models are Few-Shot Learners (Brown et al., 2020) — GPT-3	Documents the emergence of in-context learning. The paper that made it clear scale alone changes capabilities.
InstructGPT (Ouyang et al., 2022)	The RLHF recipe. The technique behind ChatGPT, Claude, Gemini, and all modern assistant models.
Chain-of-Thought Prompting (Wei et al., 2022)	Showed that asking the model to "think step by step" dramatically improves reasoning. Direct precursor to today's reasoning models.
Constitutional AI (Bai et al., 2022)	Anthropic's approach to RLHF using AI feedback alongside human feedback. The method behind Claude's training.
LoRA (Hu et al., 2021)	Parameter-efficient fine-tuning. See the companion study guide for full coverage.

This is one of several AI Engineer study guides. The companion guide on LoRA covers the dominant fine-tuning method used to customize most modern LLMs. Other guides in the series cover the foundational papers in depth.